

0.1 Theoretical Background of Model Context Protocol

0.1.1 Motivation and Design Rationale

In the era of rapid advancement in Artificial Intelligence (AI), major technology organizations are increasingly investing in the development of Large Language Models (LLMs) to support human decision-making, particularly in the field of Information Technology and Cybersecurity. However, the prevailing interaction paradigm remains largely limited to a question–answer mechanism, which fails to fully leverage the contextual reasoning capabilities of AI systems. This limitation often results in inaccurate or suboptimal outputs due to insufficient or poorly structured input context.

Furthermore, most LLMs operate in relatively isolated environments, constrained by static training datasets and lacking the ability to directly interact with real-time data sources or external tools. Integrating new data sources typically requires custom-built connectors, leading to architectural fragmentation and poor scalability. This issue becomes increasingly problematic in complex cybersecurity environments, where systems such as SIEM, SOAR, and threat intelligence platforms must be continuously integrated and updated.

To address these challenges, Anthropic introduced the Model Context Protocol, MCP is defined as a standardized framework designed to unify the interaction between AI models and external data sources or tools. MCP enables dynamic context injection and tool utilization, allowing AI systems to generate more accurate and context-aware responses.

The design philosophy of MCP is inspired by the modular architecture of the Language Server Protocol (LSP), emphasizing a clear separation between *intelligence* (LLMs) and *context* (data and tools). In traditional architectures, integrating N models with M data sources results in $N \times M$ custom adapters, significantly increasing system complexity. MCP transforms this integration model into a scalable $N + M$ architecture, thereby reducing development overhead and improving extensibility.

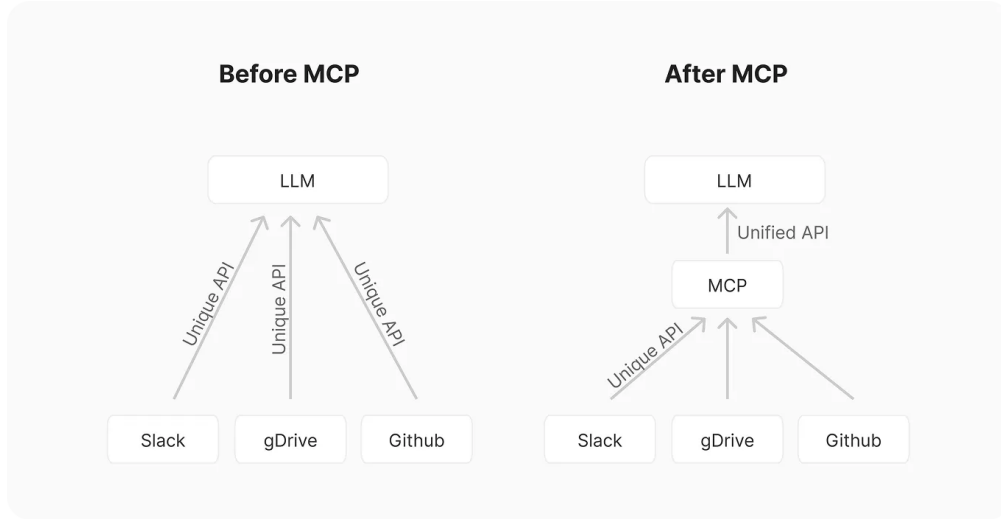


Figure 1: Comparison between traditional $N \times M$ integration and MCP's $N + M$ architecture.

0.1.2 Architecture and Core Components

The Model Context Protocol is designed as a layered and modular architecture, enabling independent development, scalability, and maintainability of each component. This architectural separation ensures that system upgrades or modifications can be performed without disrupting the overall infrastructure.

At the top layer, the **MCP Host** acts as the central orchestrator. It is responsible for managing AI agent workflows, controlling access permissions, and presenting analytical results to security operators. In the context of this project, the Host serves as the control plane for automated incident response.

The **MCP Client** functions as an intermediary communication layer. It maintains persistent connections with MCP Servers and performs capability negotiation to ensure compatibility. Additionally, the Client translates AI-generated requests into standardized JSON-RPC messages.

At the lower layer, **MCP Servers** encapsulate specific data sources or tools as lightweight wrappers. These servers process incoming requests and interact directly with backend systems such as SIEM platforms, databases, or monitoring tools. The communication between Client and Server is implemented using JSON-RPC 2.0, supporting multiple transport mechanisms including STDIO for low-latency local communication and Server-Sent Events (SSE) over HTTP for distributed deployments.

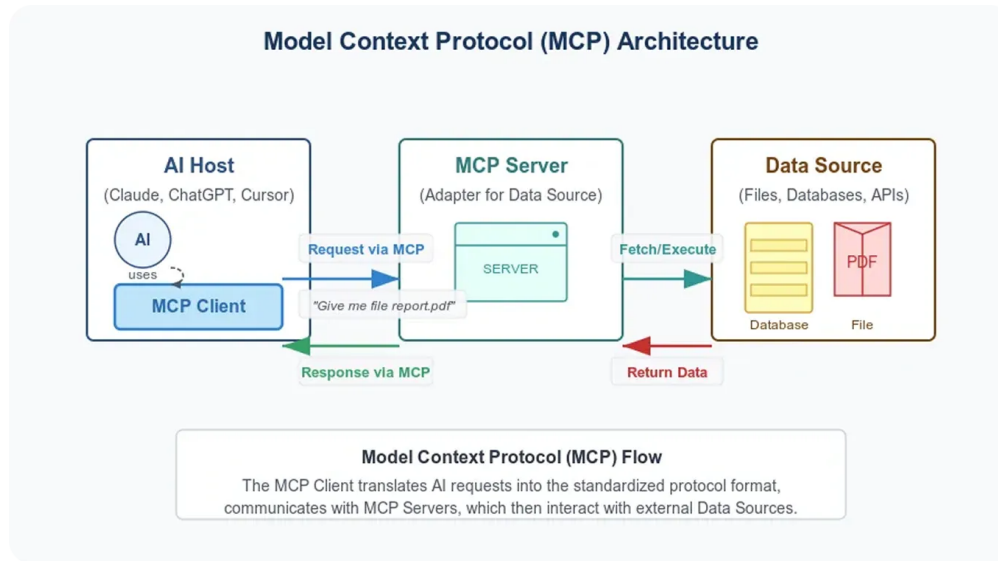


Figure 2: Layered architecture of Model Context Protocol including Host, Client, and Server components.

0.1.3 Capability Negotiation and Connection Lifecycle

A critical feature of MCP is the capability negotiation mechanism, which ensures compatibility between Clients and Servers during connection initialization.

The process begins with an `initialize` request sent from the Client to the Server. This request includes the protocol version and a list of supported capabilities. The Server then responds with its own capabilities, indicating available features such as resource subscriptions or tool execution.

The connection is established only after both parties agree on a shared subset of capabilities. A final `initialized` notification confirms that the session is ready for operation.

This adaptive negotiation mechanism ensures backward compatibility and system stability, allowing components with different versions to interoperate effectively.

0.1.4 MCP as an Intelligent Agent Framework

Model Context Protocol extends traditional function calling into a stateful, bidirectional interaction model, transforming LLMs from passive text processors into active intelligent agents.

This transformation is achieved through a set of primitives:

Server-side primitives:

- **Resources:** Provide contextual data such as logs, configurations, and system states.
- **Tools:** Enable actionable operations such as isolating endpoints or blocking malicious IPs.
- **Prompts:** Standardize investigation workflows.

Client-side primitives:

- **Sampling:** Allows Servers to request additional AI inference.
- **Elicitation:** Enables user interaction for approval or additional input.

Together, these primitives enable AI agents to perceive, reason, and act within real-world cybersecurity environments.

0.1.5 Applications in Log Normalization and OCSF Mapping

One of the major challenges in cybersecurity is the heterogeneity of log formats across different systems. MCP addresses this issue by enabling standardized data processing at the Server level.

Logs from various sources can be normalized into common schemas such as the Open Cybersecurity Schema Framework (OCSF), allowing AI models to perform consistent analysis and correlation across multiple data streams.

This capability can improve threat detection accuracy and supports more efficient incident response workflows.

0.1.6 Dynamic Tool Discovery and Context Optimization

In large-scale cybersecurity systems, the number of available tools and services can grow significantly, making it inefficient to include all tool descriptions within the context window of a Large Language Model (LLM). Excessive context loading not only increases computational overhead but also degrades response accuracy and latency.

To address this issue, MCP introduces a dynamic tool discovery mechanism, also known as search-based discovery. Instead of loading all available tools into the model context, the MCP Host selectively retrieves only the most relevant tools based on the current query or task. This selective loading strategy improves efficiency and scalability of context management.

This approach is particularly beneficial in Security Operations Center (SOC) environments, where dozens of MCP Servers may expose hundreds of tools. By dynamically selecting relevant tools, the system ensures scalability without overwhelming the AI agent.

In parallel, MCP supports a *Code Mode* execution paradigm, allowing AI agents to generate and execute scripts in Python or TypeScript within a controlled sandbox environment. Instead of processing large datasets directly through the LLM, data-intensive operations such as filtering, aggregation, and correlation are executed locally on the MCP Server.

This architectural shift minimizes data transfer, reduces latency, and enhances privacy by ensuring that sensitive data remains within the local infrastructure.

0.1.7 Security and Governance Mechanisms

Despite its powerful capabilities, MCP incorporates multiple security and governance mechanisms to ensure safe and controlled operation within sensitive environments.

One of the core mechanisms is *On-Behalf-Of (OBO) Execution*, which binds the permissions of the AI agent directly to the permissions of the authenticated user. This ensures that the AI cannot perform actions beyond the authority of the user, thereby preventing privilege escalation.

Additionally, MCP enforces a Zero Trust model through its initialization handshake process. During capability negotiation, each MCP Server explicitly declares its permissions, distinguishing between read-only operations (Resources) and state-changing actions (Tools). This strict separation prevents unauthorized modifications to critical systems.

In Code Mode, MCP introduces secure bindings to protect sensitive credentials such as API keys. These bindings ensure that secrets are never exposed to the LLM, even during execution. All sensitive operations are handled within a sandboxed environment, minimizing the risk of data leakage.

Furthermore, MCP supports secure user interaction through mechanisms such as Elicitation and external authentication flows. For example, sensitive inputs such as passwords or one-time passwords (OTP) can be collected through secure web interfaces rather than being directly processed by the AI model.

These combined mechanisms establish a robust security foundation, making MCP suitable for deployment in high-risk cybersecurity environments.

0.1.8 Role of MCP in the Proposed System

In this project, the Model Context Protocol (MCP) serves as the core communication framework that enables secure and structured interaction between the AI agent and the target system. Instead of allowing the AI model to directly access production resources, MCP provides a controlled architecture in which the Host manages orchestration while the Server performs execution on the target environment. This separation improves both security and operational reliability.

The MCP Host, implemented as a Python-based controller, acts as the central orchestrator and decision-making layer of the entire SOAR workflow. It is responsible for aggregating all incident context, including security alerts collected from Wazuh and the vulnerable source code retrieved from the target system. This centralized context is then prepared and delivered to the Claude AI model for vulnerability analysis and patch generation.

In addition, the Host coordinates the interaction between all major components of the system. It receives alerts from the SIEM layer, instructs the MCP Server to retrieve source code, sends analysis requests to the AI model, and manages the Docker sandbox used for patch validation. The Host also supervises the iterative self-correction loop by monitoring sandbox execution results. If the generated patch fails during testing, the Host forces the AI to reanalyze the issue and generate a revised patch, with the process repeating for a limited number of iterations.

From a security perspective, the MCP Host also functions as a gatekeeping mechanism. Through Telegram integration, it pauses the remediation workflow and waits for administrator approval before allowing deployment to the production system. This human-in-the-loop approach ensures that high-risk modifications are never applied without explicit authorization.

The MCP Server, deployed as a container on the target machine, acts as the execution layer and the direct interface to the production environment. It provides real system data by executing controlled operations such as reading source code files and extracting configuration information when requested by the Host or the AI agent. This allows the AI to analyze vulnerabilities using actual production context rather than incomplete assumptions.

After a patch is generated and approved, the MCP Server performs the final remediation step by replacing the vulnerable source code with the validated patched version. Before deployment, it also supports consistency verification by allowing the AI to re-read the production file one final time, ensuring that no unexpected changes occurred during the approval process.

Furthermore, if the AI agent determines that additional information is required for accurate remediation, such as configuration files, utility modules, or supporting database logic, the MCP Server acts as an on-demand knowledge provider. This allows the AI to dynamically request missing context

instead of generating unreliable assumptions.

Through this Host-Server architecture, MCP enables a secure, auditable, and context-aware remediation process, making it a critical foundation for the intelligent automated response system proposed in this project.